

CONNEXUS

Projet de Stage — 2024 / 2025

DOCUMENTATION TECHNIQUE

Plateforme d'Audit Automatisé de Domaines

Champ	Détail
Stagiaire	Nathan
Entreprise	Connexus
Période	2024 – 2025
Technologie principale	Python · FastAPI · PostgreSQL
Version du document	1.0
Date	13 mai 2026

1. Introduction

Ce document constitue la documentation technique complète de la plateforme AuditScan, développée dans le cadre d'un stage chez Connexus. Il décrit l'architecture du système, les choix technologiques, le fonctionnement de chaque module, ainsi que les procédures d'installation et de déploiement.

1.1 Contexte et objectifs

Connexus accompagne ses clients dans la sécurisation de leur présence en ligne. Dans ce cadre, la direction a identifié le besoin d'un outil automatisé capable d'analyser rapidement la posture de sécurité d'un domaine web, sans nécessiter d'intervention manuelle.

La plateforme AuditScan répond à ce besoin en proposant :

- Une analyse automatisée multi-dimensionnelle (DNS, SSL, HTTP, réseau, performance, technologies)
- Un score de sécurité global sur 100 points
- Une détection des problèmes classés par sévérité
- Une interface web moderne et accessible
- Une persistance des données en base de données PostgreSQL

1.2 Périmètre fonctionnel

Module	Fonctionnalité	Priorité
DNS	Vérification SPF, DKIM, DMARC, enregistrements A/MX/TXT	Haute
SSL/TLS	Validité du certificat, version TLS, date d'expiration	Haute
HTTP Headers	HSTS, CSP, X-Frame-Options, X-Content-Type	Haute
Ports réseau	Scan Nmap des ports ouverts, détection des ports dangereux	Haute
Performance	Temps de chargement, taille de page, score	Moyenne
Technologies	Détection CMS, frameworks JS, serveur, CDN, analytics	Moyenne

2. Architecture du système

2.1 Vue d'ensemble

L'application suit une architecture trois-tiers classique, séparant clairement la présentation, la logique métier et la persistance des données.

Architecture : Frontend HTML/CSS/JS → API REST FastAPI (Python) → Base de données PostgreSQL

Couche	Technologie	Rôle
Présentation	HTML5 / CSS3 / JavaScript	Interface utilisateur, visualisation des résultats
API / Logique métier	Python 3.12 + FastAPI	Orchestration des scans, calcul du score
Persistance	PostgreSQL 16	Stockage des résultats et historique des scans
Serveur ASGI	Uvicorn	Serveur HTTP asynchrone pour FastAPI

2.2 Structure des fichiers

```
audit-platform/
├── app/
│   ├── main.py           # Point d'entrée API, routes FastAPI
│   ├── database.py       # Connexion SQLAlchemy à PostgreSQL
│   ├── models.py        # Modèles ORM (tables BDD)
│   └── modules/
│       ├── dns_scan.py   # Module analyse DNS
│       ├── ssl_scan.py   # Module certificat SSL
│       ├── http_scan.py  # Module headers HTTP
│       ├── port_scan.py  # Module scan réseau Nmap
│       ├── perf_scan.py  # Module performance
│       ├── tech_scan.py  # Module détection technologies
│       └── scoring.py    # Calcul du score global
├── index.html            # Interface web
├── style.css             # Styles CSS
├── app.js               # Logique JavaScript
└── requirements.txt     # Dépendances Python
```

2.3 Flux de données

Lorsqu'un utilisateur soumet un domaine, voici le déroulement complet :

- L'interface JavaScript envoie une requête POST `/scan?domain=exemple.com` à l'API FastAPI
- FastAPI enregistre le domaine en base de données et crée un enregistrement de scan
- Les 6 modules s'exécutent séquentiellement et retournent leurs résultats
- Le module scoring calcule le score global sur 100 à partir des résultats
- Tous les résultats sont persistés en base de données
- L'API retourne un objet JSON complet à l'interface
- L'interface affiche les résultats avec animations et visualisations

3. Base de données

3.1 Schéma relationnel

La base de données PostgreSQL comporte 7 tables organisées autour d'une relation domaine → scan → résultats.

Table	Description	Clé principale
domains	Stocke les domaines analysés	id (SERIAL)
scans	Un enregistrement par exécution de scan	id, domain_id (FK)
results_dns	Enregistrements DNS récupérés	id, scan_id (FK)
results_ssl	Données du certificat SSL	id, scan_id (FK)
results_http	Présence ou absence des headers HTTP	id, scan_id (FK)
results_ports	Liste des ports ouverts détectés	id, scan_id (FK)
issues_detected	Problèmes de sécurité identifiés	id, scan_id (FK)

3.2 Utilité de la persistance

La base de données remplit plusieurs rôles essentiels dans la plateforme :

- Historique : conservation de tous les scans passés avec leur date
- Évolution : possibilité de comparer le score d'un domaine dans le temps
- Génération de rapports : export des données stockées vers PDF ou Excel
- Optimisation : possibilité d'éviter de rescanner un domaine récemment analysé

4. API REST

4.1 Endpoints disponibles

Méthode	Route	Description	Paramètre
POST	/scan	Lance un scan complet	domain (query string)
GET	/scan/{id}	Récupère les résultats d'un scan	id (path)
GET	/domains	Liste tous les domaines scannés	—
GET	/report/{scan_id}	Génère un rapport au format PDF	scan_id

4.2 Exemple de réponse POST /scan

```
{
  "scan_id": 42,
```

```
"domain": "exemple.com",
"score": 78,
"dns": { "SPF": true, "DKIM": false, "DMARC": true, "A": ["104.21.45.67"] },
"ssl": { "valid": true, "tls_version": "TLSv1.3", "expiry_date": "2025-11-15" },
"http": { "hsts": true, "csp": false, "x_frame": true, "x_content_type": true },
"ports": [{ "port": 443, "service": "https", "state": "open" }],
"performance": { "load_time_ms": 820, "page_size_kb": 245, "score": 80 },
"technologies": { "cms": "WordPress", "frameworks_js": ["jQuery"], "server": "Nginx" },
"issues": [{ "severity": "medium", "description": "DKIM absent" }]
}
```

5. Modules d'analyse

5.1 Module DNS (dns_scan.py)

Ce module utilise la librairie dnspython pour interroger les serveurs DNS et récupérer les enregistrements du domaine.

Enregistrement	Rôle	Impact sécurité
A	Adresses IP du serveur	Information
MX	Serveurs de messagerie	Information
TXT / SPF	Politique d'envoi d'emails autorisés	Anti-spam
TXT / DKIM	Signature cryptographique des emails	Anti-usurpation
TXT / DMARC	Politique de traitement des emails frauduleux	Critique

5.2 Module SSL (ssl_scan.py)

Ce module ouvre une connexion TLS sur le port 443 et analyse le certificat présenté par le serveur.

- Validité du certificat (date d'expiration vs date actuelle)
- Version du protocole TLS (1.2 recommandé, 1.3 optimal)
- Type de certificat (DV, OV, EV selon l'organisation)
- Nombre de jours restants avant expiration

5.3 Module HTTP Headers (http_scan.py)

Ce module effectue une requête GET et analyse les en-têtes de la réponse HTTP.

Header	Rôle	Points attribués
Strict-Transport-Security (HSTS)	Force l'utilisation de HTTPS	10 pts

Header	Rôle	Points attribués
Content-Security-Policy (CSP)	Empêche les injections XSS	10 pts
X-Frame-Options	Empêche le clickjacking	5 pts
X-Content-Type-Options	Empêche le MIME sniffing	5 pts

5.4 Module Réseau (port_scan.py)

Ce module utilise Nmap via la librairie python-nmap pour scanner les ports les plus courants du serveur cible.

- Scan rapide des ports communs (option -F de Nmap)
- Détection des services exposés (HTTP, SSH, FTP, RDP...)
- Identification des ports considérés comme dangereux : 21 (FTP), 23 (Telnet), 3389 (RDP), 5900 (VNC)

5.5 Module Technologies (tech_scan.py)

Ce module analyse le code source HTML de la page et les headers HTTP pour détecter les technologies utilisées.

Catégorie	Technologies détectées
CMS	WordPress, PrestaShop, Shopify, Wix, Squarespace, Joomla, Drupal, Magento
Frameworks JS	React, Next.js, Vue.js, Nuxt.js, Angular, jQuery, Svelte
Serveur web	Nginx, Apache, IIS, Cloudflare, Lighttpd
Langage backend	PHP (avec version), ASP.NET, Node.js, Python
CDN	Cloudflare, Fastly, Akamai, AWS CloudFront
Analytics	Google Analytics / GTM, Matomo, Plausible, Hotjar

5.6 Calcul du score (scoring.py)

Le score global est calculé sur 100 points selon la grille suivante :

Catégorie	Critère	Points
DNS (25 pts)	SPF présent	8 pts
DNS (25 pts)	DKIM présent	8 pts
DNS (25 pts)	DMARC présent	9 pts
SSL (30 pts)	Certificat valide	20 pts
SSL (30 pts)	TLS 1.3	10 pts / TLS 1.2 = 5 pts
HTTP Headers (30 pts)	HSTS présent	10 pts
HTTP Headers (30 pts)	CSP présent	10 pts

Catégorie	Critère	Points
HTTP Headers (30 pts)	X-Frame + X-Content-Type	5 pts chacun
Ports (15 pts)	Aucun port dangereux exposé	15 pts
Ports (15 pts)	1-2 ports dangereux	7 pts
Ports (15 pts)	3+ ports dangereux	0 pt

6. Installation et déploiement

6.1 Prérequis

Logiciel	Version minimale	Lien de téléchargement
Python	3.12+	python.org/downloads
PostgreSQL	15+	postgresql.org/download
Nmap	7.x	nmap.org/download.html
Node.js (optionnel)	18+	nodejs.org

6.2 Installation pas à pas (Windows)

Étape 1 — Créer l'environnement

```
mkdir C:\audit-platform
cd C:\audit-platform
python -m venv venv
venv\Scripts\activate
```

Étape 2 — Installer les dépendances Python

```
pip install fastapi uvicorn psycpg2-binary sqlalchemy dnspython
pip install requests python-nmap cryptography python-dotenv
```

Étape 3 — Créer les tables PostgreSQL

Dans pgAdmin, sélectionner la base audit_platform puis exécuter le script SQL de création des tables (voir Annexe A).

Étape 4 — Lancer le serveur

```
cd C:\audit-platform
venv\Scripts\activate
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

L'API est accessible sur <http://localhost:8000> · La documentation Swagger est disponible sur <http://localhost:8000/docs>

6.3 Déploiement sur un autre poste

Pour partager le projet sur un autre PC, fournir l'ensemble du code source (sans le dossier venv/) et suivre les étapes suivantes :

- Installer Python, PostgreSQL et Nmap
- Recréer l'environnement virtuel avec `python -m venv venv`
- Réinstaller les dépendances avec `pip install -r requirements.txt`
- Mettre le mot de passe défini sur PgAdmin dans le fichier `database.py`
- Recréer les tables en base de données
- Lancer le serveur avec `uvicorn`

7. Considérations de sécurité

7.1 Sécurité de l'application

Risque	Mesure mise en place
Injection SQL	Utilisation de SQLAlchemy ORM (requêtes paramétrées)
CORS	Middleware FastAPI configuré (à restreindre en production)
Timeout des requêtes	Timeout de 10-15s sur toutes les requêtes externes
Erreurs SSL ignorées	<code>verify=False</code> uniquement pour les modules d'analyse (pas l'API)

7.2 Limitations connues

- Le scan Nmap nécessite des droits administrateur sur certains systèmes
- Le mode `verify=False` dans le module HTTP peut exposer à des attaques MITM lors du scan
- Le CORS est configuré en mode permissif (`allow_origins=["*"]`) pour le développement

8. Dépendances et licences

Librairie	Version	Licence	Rôle
FastAPI	0.111+	MIT	Framework API REST

Librairie	Version	Licence	Rôle
Uvicorn	0.30+	BSD	Serveur ASGI
SQLAlchemy	2.0+	MIT	ORM base de données
psycopg2-binary	2.9+	LGPL	Driver PostgreSQL
dnspython	2.6+	ISC	Requêtes DNS
requests	2.31+	Apache 2.0	Requêtes HTTP
python-nmap	0.7+	GPL	Interface Nmap
cryptography	42+	Apache/BSD	Analyse SSL
python-dotenv	1.0+	BSD	Variables d'environnement
Alembic	1.13+	MIT	Migrations base de données
Pydantic	2.6+	MIT	Validation et sérialisation des données

Annexe A — Script SQL de création des tables

```
CREATE DATABASE audit_platform;

-- Connexion à la base audit_platform avant d'exécuter la suite

CREATE TABLE domains (
    id SERIAL PRIMARY KEY,
    domain VARCHAR(255) NOT NULL,
    date_creation TIMESTAMP DEFAULT NOW()
);

CREATE TABLE scans (
    id SERIAL PRIMARY KEY,
    domain_id INTEGER REFERENCES domains(id),
    date_scan TIMESTAMP DEFAULT NOW()
);

CREATE TABLE results_dns (
    id SERIAL PRIMARY KEY,
    scan_id INTEGER REFERENCES scans(id),
    record_type VARCHAR(50),
    value TEXT
);

CREATE TABLE results_ssl (
    id SERIAL PRIMARY KEY,
    scan_id INTEGER REFERENCES scans(id),
    valid BOOLEAN, expiry_date TIMESTAMP,
    cert_type VARCHAR(100), tls_version VARCHAR(50)
);
```

```
CREATE TABLE results_http (  
    id SERIAL PRIMARY KEY,  
    scan_id INTEGER REFERENCES scans(id),  
    hsts BOOLEAN, csp BOOLEAN,  
    x_frame BOOLEAN, x_content_type BOOLEAN  
);  
  
CREATE TABLE results_ports (  
    id SERIAL PRIMARY KEY,  
    scan_id INTEGER REFERENCES scans(id),  
    port INTEGER, service VARCHAR(100), state VARCHAR(50)  
);  
  
CREATE TABLE issues_detected (  
    id SERIAL PRIMARY KEY,  
    scan_id INTEGER REFERENCES scans(id),  
    severity VARCHAR(50), description TEXT  
);
```